

MIIA0106

PYTHON AND C PROGRAMMING LANGUAGE

ข้อมูลเชิงโครงสร้าง (Struct) และ Typedef

อาจารย์ สุทิศ องอาจ

LEC 8



ตารางสอนรายวิชา MIIA0106

Section A

วัน	คาบ	เวลาเรียน	ห้อง
อังคาร (TUE)	คาบ 2	12.30 – 15.00 น.	D604
อังคาร (TUE)	คาบ 3	15.30 – 18.00 น.	MII203

Section B

วัน	คาบ	เวลาเรียน	ห้อง
เสาร์ (SAT)	คาบ 4	16.00 – 18.30 น.	MII207B
อาทิตย์ (SUN)	คาบ 4	15.30 – 17.30 น.	MII201

ช่วงสอบ กลางภาค 10 ม.ค – 18 ม.ค 69

สอบประจำภาค วันอาทิตย์ ที่ 15 มีนาคม พ.ศ. 2569
เช้า 09:00-12:00 น.

ขาดสอบ จะได้เป็น **FE**

MIIA0106

Python and C Programming Language

ข้อมูลเชิงโครงสร้าง (Struct) และ Typedef

อาจารย์ สุทิศ องอาจ

sutit@mut.ac.th

ที่มาของปัญหา (The Problem)

การเก็บข้อมูลแบบเดิม (Parallel Arrays)

สมมติเราต้องการเก็บข้อมูลนักศึกษา 5 คน ประกอบด้วย
รหัส, ชื่อเล่น, Line ID และเบอร์โทร

```
string studentIDs[5];  
string nicknames[5];  
string lineIDs[5];  
string phoneNumbers[5];
```

					
รหัสด (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

⚠ **ปัญหา:** ข้อมูลกระจัดกระจาย หากมีการเรียงลำดับ
(Sort) อาเรย์หนึ่ง ก็ต้องตามไปเรียงอาเรย์อื่นให้ตรงกัน
ซึ่งจัดการได้ยาก

ตัวอย่างที่ 1 (C++) : รับข้อมูลนักศึกษา 5 คน

แล้วแสดงผลแนวคิดตามภาพคือ Parallel Arrays (เก็บข้อมูลคนเดียวกันไว้ index เดียวกัน)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 5;

    string studentIDs[SIZE];
    string nicknames[SIZE];
    string lineIDs[SIZE];
    string phoneNumbers[SIZE];

    // รับข้อมูล
    for (int i = 0; i < SIZE; i++) {
        cout << "=== Student " << i + 1 << " ===" << endl;

        cout << "Student ID: ";
        cin >> studentIDs[i];

        cout << "Nickname: ";
        cin >> nicknames[i];

        cout << "Line ID: ";
        cin >> lineIDs[i];

        cout << "Phone Number: ";
        cin >> phoneNumbers[i];

        cout << endl;
    }

    return 0;
}
```



รคต (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

// แสดงผล

```
cout << "\n==== Student Information ==== \n";
for (int i = 0; i < SIZE; i++) {
    cout << "Student " << i + 1 << endl;
    cout << "ID : " << studentIDs[i] << endl;
    cout << "Nickname : " << nicknames[i] << endl;
    cout << "Line ID : " << lineIDs[i] << endl;
    cout << "Phone : " << phoneNumbers[i] << endl;
    cout << "----- \n";
}
```

ตัวอย่างที่ 2 : แสดงผลเรียงตาม ID จากน้อย → มาก

(Parallel Arrays)แนวคิดคือ **ต้องสลับข้อมูลทุก array พร้อมกัน** เมื่อมีการเรียง ใช้ **Bubble Sort** (เข้าใจง่าย)

```
// ===== Sort ตาม ID (น้อย → มาก) =====  
for (int i = 0; i < SIZE - 1; i++) {  
    for (int j = 0; j < SIZE - 1 - i; j++) {  
        if (studentIDs[j] > studentIDs[j + 1]) {  
            // สลับ ID  
            swapString(studentIDs[j], studentIDs[j + 1]);  
            // สลับข้อมูลที่เกี่ยวข้องทั้งหมด  
            swapString(nicknames[j], nicknames[j + 1]);  
            swapString(lineIDs[j], lineIDs[j + 1]);  
            swapString(phoneNumbers[j], phoneNumbers[j + 1]);  
        }  
    }  
}
```

```
// สามารถใช้ swap ที่เป็น library ได้เลย แต่ขอเขียนฟังก์ชัน swap เองเพื่อความเข้าใจ  
void swapString(string& a, string& b) {  
    string temp = a;  
    a = b;  
    b = temp;  
}
```



รคต (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

ที่มาของปัญหา (The Problem)

การเก็บข้อมูลแบบเดิม (Parallel Arrays)

สมมติเราต้องการเก็บข้อมูลนักศึกษา 5 คน ประกอบด้วย
รหัส, ชื่อเล่น, Line ID และเบอร์โทร

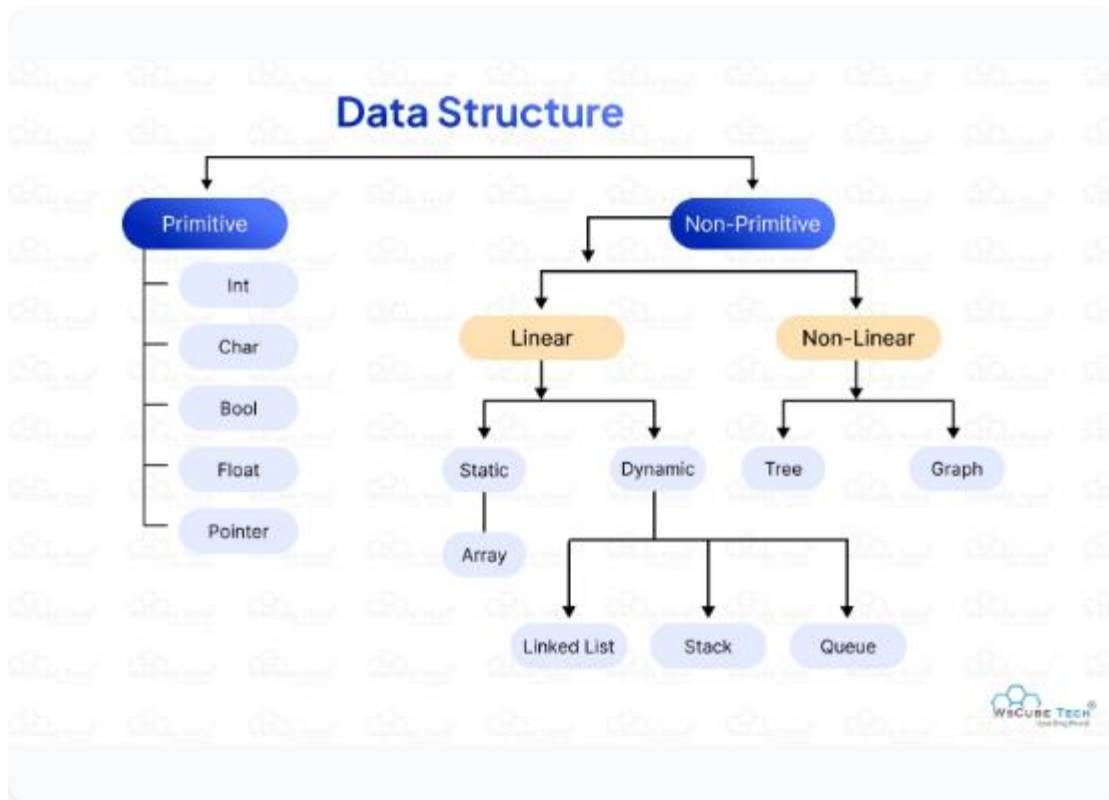
```
string studentIDs[5];  
string nicknames[5];  
string lineIDs[5];  
string phoneNumbers[5];
```

⚠ **ปัญหา:** ข้อมูลกระจัดกระจาย หากมีการเรียงลำดับ
(Sort) อารีย์หนึ่ง ก็ต้องตามไปเรียงอารีย์อื่นให้ตรงกัน
ซึ่งจัดการได้ยาก



Structured Data vs Unstructured Data: How Do They Differ?

ข้อมูลแบบโครงสร้างคืออะไร?



Structure (struct)

คือกลุ่มข้อมูลที่ประกอบด้วยสมาชิก (Members) ซึ่งสามารถเป็นข้อมูล "ต่างชนิดกัน" ได้

- > เปรียบเสมือน "กล่อง" หรือ "แบบฟอร์ม" ที่รวมข้อมูลที่เกี่ยวข้องกันไว้ในที่เดียว
- > ต่างจาก Array ที่สมาชิกทุกตัวต้องเป็นชนิดเดียวกัน
- > ช่วยให้การจัดการข้อมูลมีความเป็นระเบียบและเข้าใจง่ายขึ้น

การประกาศและใช้งาน

</>การสร้าง Struct

```
struct Student {  
    string studentID;  
    string nickname;  
    string lineID;  
    string phoneNumber;  
};
```

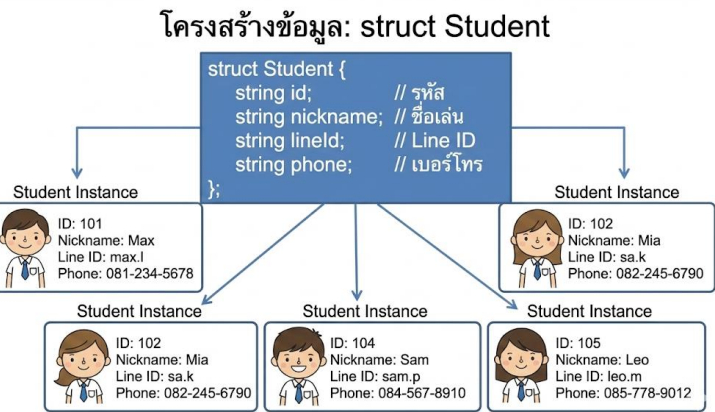
เป็นการกำหนดแม่แบบ (Template) ของข้อมูล

≡การนำไปใช้

```
int main() {  
    // สร้าง Array ของ Struct ขนาด 5  
    Student students[5];  
}
```

เราจะได้ตัวแปรกลุ่ม students ที่เก็บข้อมูลนักศึกษาได้ 5 คน โดยแต่ละคนมีข้อมูลครบทั้ง 4 อย่าง

ประกาศตัวแปรโครงสร้างทันที



```
// กำหนดโครงสร้าง struct  
struct Student {  
    string studentID;  
    string nickname;  
    string lineID;  
    string phoneNumber;  
};  
  
int main() {  
    Student students[5]; // สร้างอาร์เรย์ของ struct ขนาด 5  
}
```

```
struct Student {  
    string studentID;  
    string nickname;  
    string lineID;  
    string phoneNumber;  
} students[5]; // ประกาศตัวแปรโครงสร้างทันที
```

					
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

การเข้าถึงและใช้งานข้อมูล



Dot Operator (.)

ใช้เครื่องหมายจุด เพื่อเข้าถึง
สมาชิกภายใน Struct

```
struct_var.member
```



การรับค่า (Input)

รับค่าเข้าไปเก็บในตัวแปรย่อย
แต่ละตัว

```
cin >> students[i].studentID;
```



การแสดงผล (Output)

ดึงค่าจากตัวแปรย่อยมา
แสดงผล

```
cout << students[i].nickname;
```

แบบที่ 1 : ตัวแปรเดี่ยวของ struct (Single struct)

```
Student s1;
```

```
s1.studentID = "101";
```

```
s1.nickname = "Max";
```

แบบที่ 2 : Array ของ struct (ใช้บ่อยมาก)

```
Student students[5];
```

```
students[0].studentID = "101";
```

แบบที่ 3 : ส่ง struct เข้า–ออกฟังก์ชัน

```
void printStudent(Student s) {  
    cout << s.studentID << endl;  
}
```

หรือ (แบบแก้ค่าได้)

```
void updateStudent(Student &s) {  
    s.nickname = "NewName";  
}
```

แบบที่ 4 : Array ของ struct + ฟังก์ชัน (ระดับใช้งานจริง)

```
void sortByID(Student students[], int size) {  
    for (int i = 0; i < size - 1; i++) {  
        for (int j = 0; j < size - 1 - i; j++) {  
            if (students[j].id > students[j + 1].id) {  
                swapStudent(students[j], students[j + 1]);  
            }  
        }  
    }  
}
```

แบบที่ 5 Pointer ชี้ไปที่ struct เดียว

ประกาศ

```
Student s1;  
Student *p; // pointer to struct
```

ชี้ pointer ไปที่ตัวแปร

```
p = &s1;
```

เข้าถึงสมาชิก

```
(*p).studentID = "101";  
(*p).nickname = "Max";
```

-> = shortcut ของ (*p).

```
p->studentID = "101";  
p->nickname = "Max";
```


การจัดการข้อความ (Strings)

แบบดั้งเดิม (C-Style char array)

- > ประกาศ: `char name[20];`
- > ต้องใช้ฟังก์ชัน `strcpy` ในการกำหนดค่า
- > ใช้เครื่องหมาย = โดยตรงไม่ได้

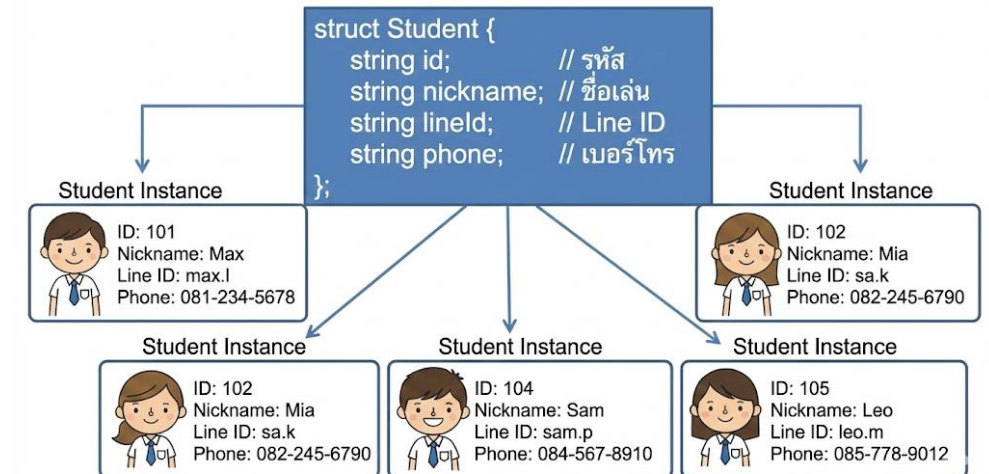
แบบใหม่ (C++ `std::string`)

- > ประกาศ: `string name;`
- > ใช้งานง่าย ยืดหยุ่นกว่า
- > สามารถกำหนดค่าด้วย = ได้เลย

โครงสร้าง Struct และ Array

					
รศด (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam		leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างข้อมูล: struct Student



ตัวอย่างที่ 3 : ใช้ struct Student → sort ง่าย ดูสะอาด และปลอดภัย

(เป็นบทสรุปจากปัญหา Parallel Arrays)

					
รหัสด (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

```
struct Student {  
    string id;  
    string nickname;  
    string lineId;  
    string phone;  
};
```

```
const int SIZE = 5;  
Student students[SIZE];  
  
// รับข้อมูล  
for (int i = 0; i < SIZE; i++) {  
    cout << "=== Student " << i + 1 << " ===" << endl;  
    cout << "Student ID: ";  
    cin >> students[i].id;  
  
    cout << "Nickname: ";  
    cin >> students[i].nickname;  
  
    cout << "Line ID: ";  
    cin >> students[i].lineId;  
  
    cout << "Phone Number: ";  
    cin >> students[i].phone;  
    cout << endl;  
}
```

```
const int SIZE = 5;  
  
string studentIDs[SIZE];  
string nicknames[SIZE];  
string lineIDs[SIZE];  
string phoneNumbers[SIZE];  
  
// รับข้อมูล  
for (int i = 0; i < SIZE; i++) {  
    cout << "=== Student " << i + 1 << " ===" << endl;  
  
    cout << "Student ID: ";  
    cin >> studentIDs[i];  
  
    cout << "Nickname: ";  
    cin >> nicknames[i];  
  
    cout << "Line ID: ";  
    cin >> lineIDs[i];  
  
    cout << "Phone Number: ";  
    cin >> phoneNumbers[i];  
  
    cout << endl;  
}
```

ตัวอย่างที่ 3 : ใช้ struct Student → sort ง่าย ดูสะอาด และปลอดภัย

(เป็นบทสรุปจากปัญหา Parallel Arrays)

					
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-345-6789	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

```
// sort ตาม ID (น้อย → มาก)
for (int i = 0; i < SIZE - 1; i++) {
    for (int j = 0; j < SIZE - 1 - i; j++) {
        if (students[j].id > students[j + 1].id) {
            swapStudent(students[j], students[j + 1]); //1 บรรทัด
        }
    }
}
```

```
// ===== Sort ตาม ID (น้อย → มาก) =====
for (int i = 0; i < SIZE - 1; i++) {
    for (int j = 0; j < SIZE - 1 - i; j++) {
        if (studentIDs[j] > studentIDs[j + 1]) {
            // สลับ ID
            swapString(studentIDs[j], studentIDs[j + 1]);
            // สลับข้อมูลที่เกี่ยวข้องทั้งหมด
            swapString(nicknames[j], nicknames[j + 1]);
            swapString(lineIDs[j], lineIDs[j + 1]);
            swapString(phoneNumbers[j], phoneNumbers[j + 1]);
        }
    }
}
```

```
// สามารถใช้ swap ที่เป็น library ได้เลย แต่ขอเขียนฟังก์ชัน swap เองเพื่อความเข้าใจ
void swapString(string& a, string& b) {
    string temp = a;
    a = b;
    b = temp;
}
```

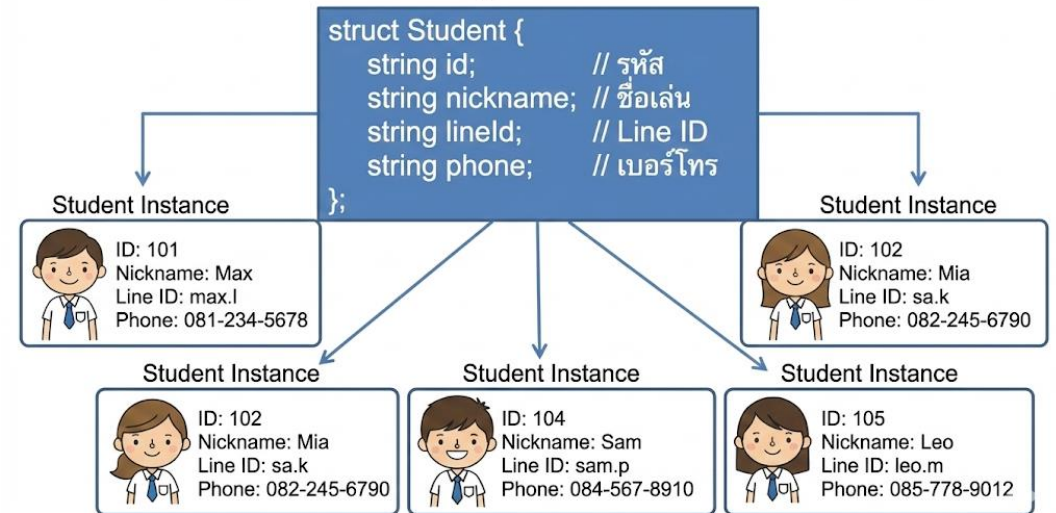
ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)

					
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างภาพรวม

- 1) `struct Student` → โครงสร้างข้อมูล
- 2) `inputStudents()` → รับข้อมูล
- 3) `sortByID()` → เรียง
- 4) `IDprintStudents()` → แสดงผล
- 5) `main()` → ควบคุมการทำงาน

โครงสร้างข้อมูล: struct Student



ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)

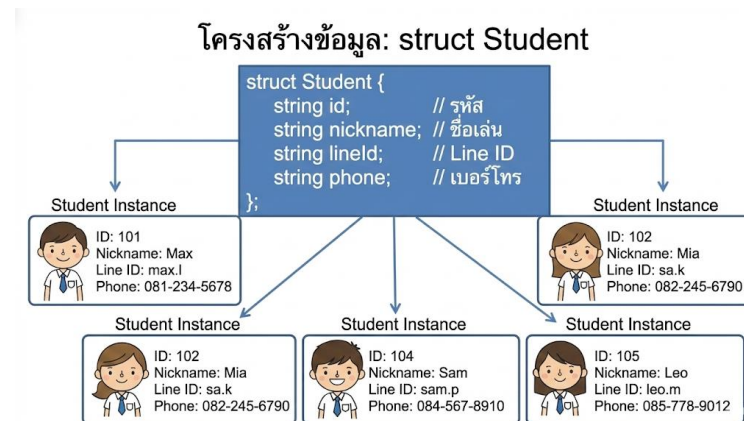
					
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างภาพรวม

- 1) **struct Student** → โครงสร้างข้อมูล
- 2) `inputStudents()` → รับข้อมูล
- 3) `sortByID()` → เรียง
- 4) `IDprintStudents()` → แสดงผล
- 5) `main()` → ควบคุมการทำงาน

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string id;
    string nickname;
    string lineId;
    string phone;
};
```



ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)

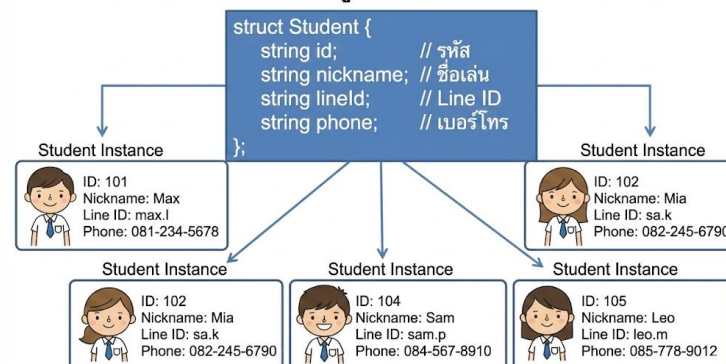
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	sa.k	sam.p	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างภาพรวม

- 1) struct Student → โครงสร้างข้อมูล
- 2) inputStudents() → รับข้อมูล
- 3) sortByID() → เรียง
- 4) IDprintStudents() → แสดงผล
- 5) main() → ควบคุมการทำงาน

```
void inputStudents(Student students[], int size) {  
    for (int i = 0; i < size; i++) {  
        cout << "=== Student " << i + 1 << " ===" << endl;  
        cout << "Student ID: ";  
        cin >> students[i].id;  
  
        cout << "Nickname: ";  
        cin >> students[i].nickname;  
  
        cout << "Line ID: ";  
        cin >> students[i].lineId;  
  
        cout << "Phone Number: ";  
        cin >> students[i].phone;  
        cout << endl;  
    }  
}
```

โครงสร้างข้อมูล: struct Student



ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)

					
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	Sa.k	sam.p	082-245-6789	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างภาพรวม

- 1) struct Student → โครงสร้างข้อมูล
- 2) inputStudents() → รับข้อมูล
- 3) sortByID() → เรียง
- 4) IDprintStudents() → แสดงผล
- 5) main() → ควบคุมการทำงาน

```
void sortByID(Student students[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - 1 - i; j++) {
            if (students[j].id > students[j + 1].id) {
                swapStudent(students[j], students[j + 1]);
            }
        }
    }
}

void swapStudent(Student &a, Student &b) {
    Student temp = a;
    a = b;
    b = temp;
}
```


ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)

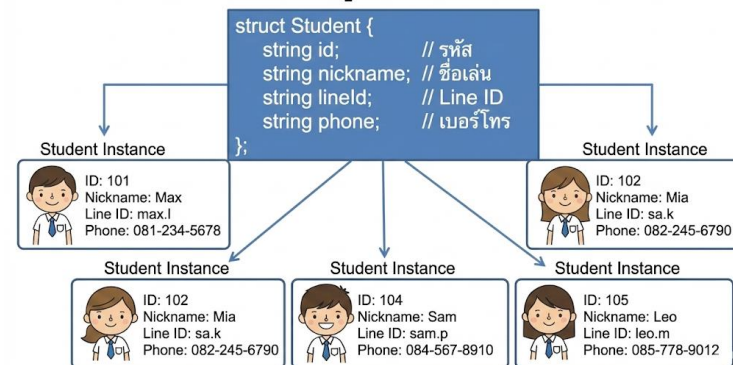
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร	Parallel Arrays
101	Max	max.l	max.l	081-234-5678	
102	Mia	sa.k	sa.k	082-245-6790	
104	Sam	sam.p	leo.m	084-567-8910	
105			joy.s	085-778-9012	

โครงสร้างภาพรวม

- 1) struct Student → โครงสร้างข้อมูล
- 2) inputStudents() → รับข้อมูล
- 3) sortByID() → เรียง
- 4) IDprintStudents() → แสดงผล
- 5) main() → ควบคุมการทำงาน

```
void printStudents(Student students[], int size) {  
    for (int i = 0; i < size; i++) {  
        cout << "ID          : " << students[i].id << endl;  
        cout << "Nickname : " << students[i].nickname << endl;  
        cout << "Line ID  : " << students[i].lineId << endl;  
        cout << "Phone    : " << students[i].phone << endl;  
        cout << "-----\n";  
    }  
}
```

โครงสร้างข้อมูล: struct Student



ตัวอย่างที่ 4 : Array of struct + แยกฟังก์ชันครบรูปแบบ (ระดับใช้งานจริง)



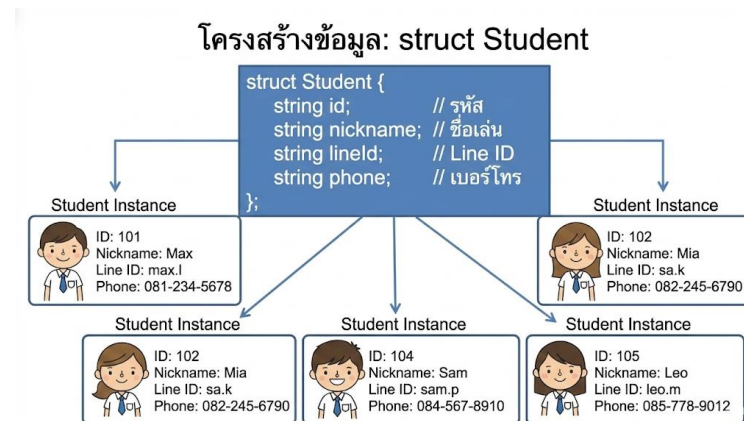
รหัส (ID)	ชื่อเล่น	Nickname	Line ID	เบอร์โทร
101	Max	max.l	max.l	081-234-5678
102	Mia	sa.k	sa.k	082-245-6790
104	Sam	sa.k	leo.m	084-567-8910
105		sa.k	joy.s	085-778-9012

Parallel Arrays

โครงสร้างภาพรวม

- 1) struct Student → โครงสร้างข้อมูล
- 2) inputStudents() → รับข้อมูล
- 3) sortByID() → เรียง
- 4) IDprintStudents() → แสดงผล
- 5) main() → ควบคุมการทำงาน

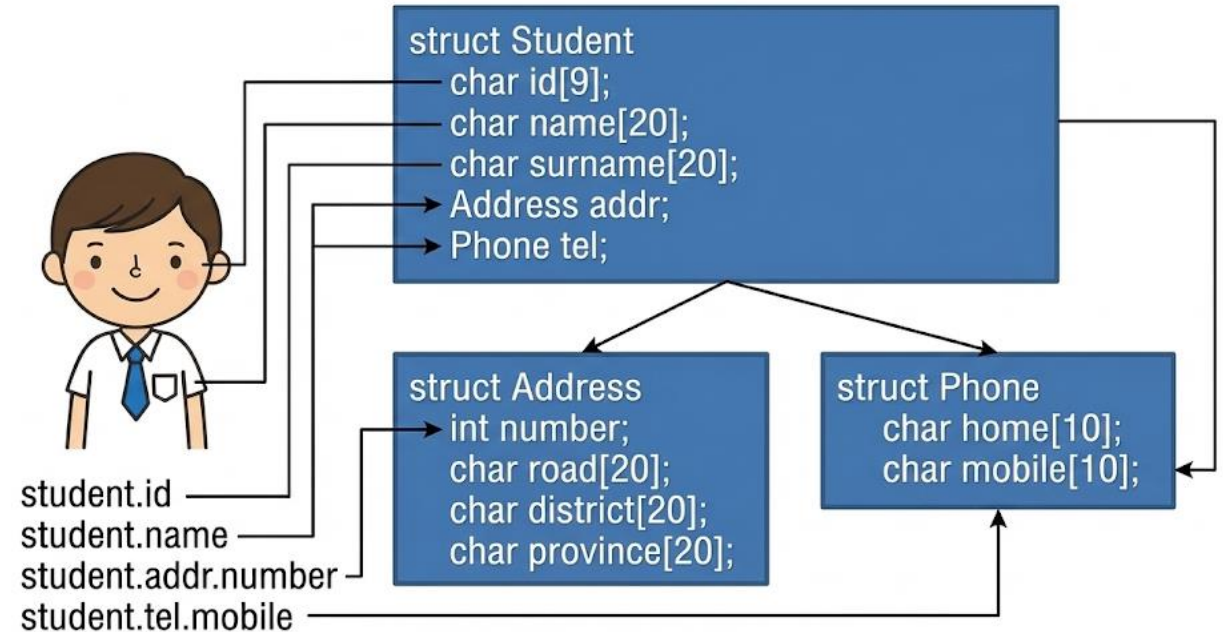
```
int main() {  
    const int SIZE = 5;  
    Student students[SIZE];  
  
    inputStudents(students, SIZE);  
  
    sortByID(students, SIZE);    // เรียงจาก ID น้อย → มาก  
  
    cout << "\n==== Student List (Sorted by ID) =====\n";  
    printStudents(students, SIZE);  
  
    return 0;  
}
```



โครงสร้างซ้อนโครงสร้าง (Nested Structures)

เราสามารถประกาศ Struct ซ้อนอยู่ภายใน Struct อีก
ตัวหนึ่งได้ เพื่อจัดหมวดหมู่ข้อมูลให้ละเอียดยิ่งขึ้น

โครงสร้างซ้อนโครงสร้าง (Nested Structures)

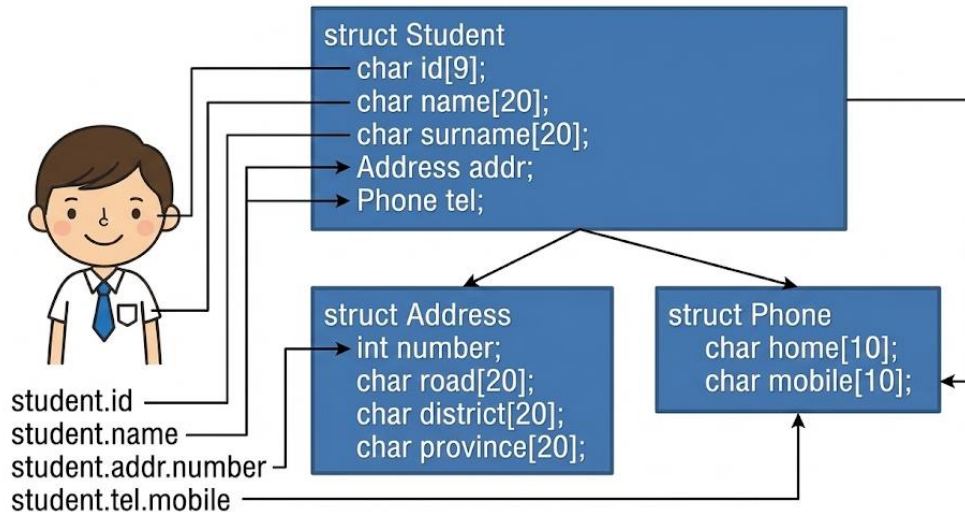


ตัวอย่างที่ 5 การใช้งาน struct ซ้อน struct (Student + Address + Phone)

โครงสร้างภาพรวม

- 1) โครงสร้างข้อมูล (Struct Design)
- 2) ฟังก์ชันรับข้อมูลนักศึกษา
- 3) ฟังก์ชันแสดงผลข้อมูลนักศึกษา
- 4) main()

โครงสร้างซ้อนโครงสร้าง (Nested Structures)



```
#include <iostream>
using namespace std;

// -----
// โครงสร้าง Address
// -----
struct Address {
    int number;
    char road[20];
    char district[20];
    char province[20];
};

// -----
// โครงสร้าง Phone
// -----
struct Phone {
    char home[10];
    char mobile[10];
};

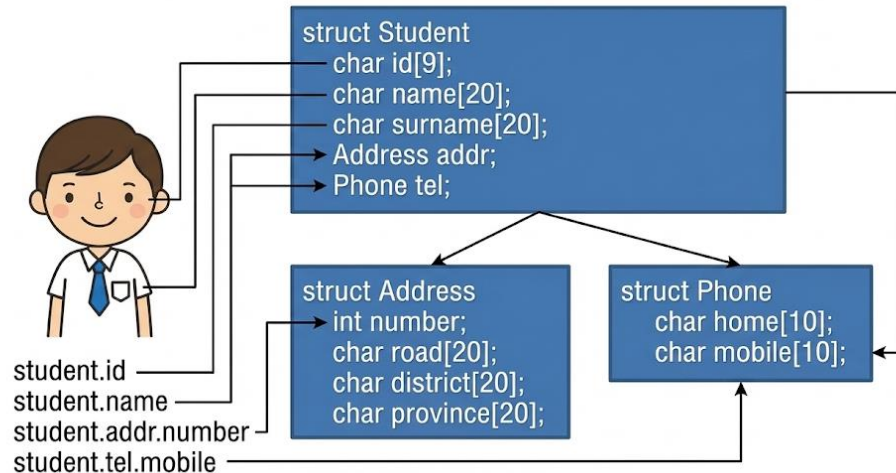
// -----
// โครงสร้าง Student
// -----
struct Student {
    char id[9];
    char name[20];
    char surname[20];
    Address addr;    // struct ซ้อน struct
    Phone tel;       // struct ซ้อน struct
};
```

ตัวอย่างที่ 5 การใช้งาน struct ซ้อน struct (Student + Address + Phone)

โครงสร้างภาพรวม

- 1) โครงสร้างข้อมูล (Struct Design)
- 2) ฟังก์ชันรับข้อมูลนักศึกษา
- 3) ฟังก์ชันแสดงผลข้อมูลนักศึกษา
- 4) main()

โครงสร้างซ้อนโครงสร้าง (Nested Structures)



```
void inputStudent(Student &student) {
    cout << "Enter Student Information\n";

    cout << "ID: ";
    cin >> student.id;

    cout << "Name: ";
    cin >> student.name;

    cout << "Surname: ";
    cin >> student.surname;

    cout << "Address - House Number: ";
    cin >> student.addr.number;

    cout << "Address - Road: ";
    cin >> student.addr.road;

    cout << "Address - District: ";
    cin >> student.addr.district;

    cout << "Address - Province: ";
    cin >> student.addr.province;

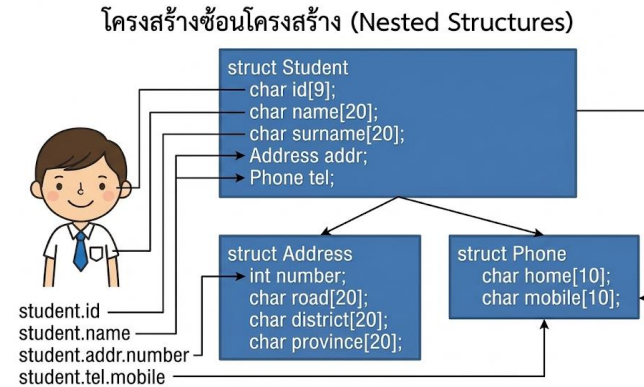
    cout << "Phone - Home: ";
    cin >> student.tel.home;

    cout << "Phone - Mobile: ";
    cin >> student.tel.mobile;
}
```

ตัวอย่างที่ 5 การใช้งาน struct ซ้อน struct (Student + Address + Phone)

โครงสร้างภาพรวม

- 1) โครงสร้างข้อมูล (Struct Design)
- 2) ฟังก์ชันรับข้อมูลนักศึกษา
- 3) ฟังก์ชันแสดงผลข้อมูลนักศึกษา
- 4) `main()`



```
void printStudent(const Student &student) {
    cout << "\n==== Student Information =====\n";
    cout << "ID      : " << student.id << endl;
    cout << "Name    : " << student.name << " " << student.surname << endl;

    cout << "Address : " << student.addr.number << ", "
         << student.addr.road << ", "
         << student.addr.district << ", "
         << student.addr.province << endl;

    cout << "Phone   : Home " << student.tel.home
         << ", Mobile " << student.tel.mobile << endl;
}
```

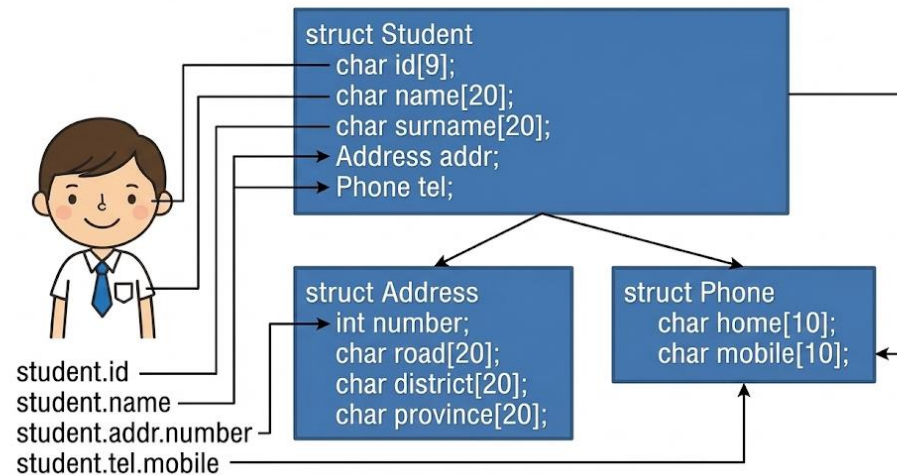
ตัวอย่างที่ 5 การใช้งาน struct ซ้อน struct (Student + Address + Phone)

โครงสร้างภาพรวม

- 1) โครงสร้างข้อมูล (Struct Design)
- 2) ฟังก์ชันรับข้อมูลนักศึกษา
- 3) ฟังก์ชันแสดงผลข้อมูลนักศึกษา
- 4) `main()`

```
int main() {  
    Student student;  
  
    inputStudent(student);  
    printStudent(student);  
  
    return 0;  
}
```

โครงสร้างซ้อนโครงสร้าง (Nested Structures)



ตัวอย่างที่ 6 (Single struct + pointer)

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string studentID;
    string nickname;
};

int main() {
    Student s1;
    Student *p = &s1;

    p->studentID = "101";
    p->nickname = "Max";

    cout << "ID: " << p->studentID << endl;
    cout << "Nickname: " << p->nickname << endl;

    return 0;
}
```


Typedef และ Enumeration

Typedef

ใช้สำหรับตั้ง "ชื่อเล่น" (Alias) ให้กับชนิดข้อมูลที่มีอยู่แล้ว เพื่อให้อ่านง่ายขึ้นหรือลดความยาวของโค้ด

```
typedef unsigned int uint;  
uint age = 20;
```

Enumeration (Enum)

ใช้สร้างชนิดข้อมูลที่มีค่าเป็นกลุ่มค่าคงที่ ช่วยให้โค้ดสื่อความหมายได้ดีกว่าการใช้ตัวเลข

```
enum class Color { RED, GREEN };  
Color c = Color::RED;
```



Sutit Ongart
Sutit@mut.ac.th
sutit.ongart@gmail.com

END